

JavaScript – Objects

Tvorba internetových aplikací

RNDr. Richard Ostertág, PhD.

KI FMFI UK Bratislava

ostertag@dcs.fmph.uniba.sk

Sources

- Douglas Crockford: “JavaScript: The Good Parts”
 - <http://www.crockford.com/javascript/>
- Mozilla Developer Network: JavaScript
 - <https://developer.mozilla.org/en-US/docs/JavaScript>
- Annotated ECMAScript 5.1
 - <http://es5.github.com/>
- ECMAScript Language Specification
 - Standard ECMA-262 / 5.1 Edition / June 2011
 - <http://ecma-international.org/ecma-262/5.1/>

OOP in Java vs. JavaScript 1/2

Class based languages (e.g. Java)	Prototype based languages (e.g. JavaScript)
class and instances are distinct entities • class is a data type • objects (as values) are instances of some class	• there is no notion of class • just constructor functions • create only instances • objects are dictionaries HashMap of ordinal types to anything
• class is defined with a class definition • instantiate a class with constructor to create object	define and create object by constructor functions
class hierarchy is created using class definitions to define subclasses	object hierarchy is created by assigning an object as the prototype associated with a constructor function

OOP in Java vs. JavaScript 2/2

Class based languages (e.g. Java)	Prototype based languages (e.g. JavaScript)
fields and methods are inherited by following the class hierarchy chain	properties and methods are inherited by following the prototype hierarchy chain
• classes specifies all fields and methods a instance of the class can have • can not add fields and methods dynamically at runtime	• constructor function or prototype specifies only initial set of properties and methods. • can add or remove properties and methods dynamically to individual objects or to the entire family of objects

OOP – classically inspired syntax

- JavaScript use a prototypal inheritance
 - objects inherit properties directly from other objects
 - the language is class-free
- prototypal inheritance is expressive but is not widely understood
- JavaScript offers an object-making syntax that is reminiscent of the classical languages
- that is the worst of both worlds
 - classical programmers are not satisfied with prototypal inheritance
 - classically inspired syntax obscures the language's true prototypal nature

JavaScript objects

- object is a container of properties with a hidden link to a prototype object
 - property has a name and a value
 - hidden link is `__proto__` property
- property name can be any string
 - **including the empty string**
 - all other values are coerced to string
 - including the **undefined**, numbers, functions ...
- property value can be any JavaScript value
 - **including the undefined**

Object literals 1/2

- provide a very convenient notation for creating new object values (objects are linked to `Object.prototype`)
- `var empty_object = {};`
- `var student = {
 name : "Juraj",
 "age y" : 20,
 "sex" : undefined
};`
- `student["age y"] ↦ 20`
- `student.sex ↦ undefined`
- `student[""] = Object.is`
- `student[Object.is] = true`
- `student ↦ Object {
 name: "Juraj", age y: 20,
 sex: undefined, : is(),
 function is() {
 [native code]
 }:
 true }`
- `student["function is() {\n [native code]\n}"] ↦ true`

Object literals 2/2

- new properties are created by assignment to nonexistent property
- quotes around a property's name are optional
 - if the name would be a legal JavaScript name and not a reserved word
- `student[{a:10}] = 7`
 - `Object { ..., [object Object]: 7 }`
- object can be nested
 - `student[-1] = {a:10}`
 - `student["-1"].a ↦ 10`

Object literals – retrieval idioms 1/2

- **student.name** or **student["name"]**
 - first notation is preferred
 - more compact and reads better
- nonexistent key returns undefined value:
 - student.foo $\mapsto$ undefined
- problem with distinguishing nonexistent key and key with undefined value:
 - student.foo === undefined $\mapsto$ true
 - student.sex === undefined $\mapsto$ true
 - use student.hasOwnProperty("foo") $\mapsto$ false (details later)

Object literals – retrieval idioms 2/2

- use the `||` operator to fill in default values
 - `student.foo || "default" ↦ "default"`
- attempting to retrieve values from undefined will throw a `TypeError` exception
- use the `&&` operator to solve this problem
 - `student.foo.bar ↦ TypeError: student.foo is undefined`
 - `student.foo && student.foo.bar ↦ undefined`
 - `student.foo = { bar: "baz" }`
 - `student.foo && student.foo.bar ↦ "baz"`

Object passing

- objects are always passed by reference
- `var marek = student;`
- `marek.name = "Marek";`
- `student.name` $\mapsto$ "Marek"
- `function f(obj) { obj.name = "***"; }`
- `f(student)`
- `marek.name` $\mapsto$ "***"
- `a = {}, b = {}, c = {};`
- `a`, `b`, and `c` all refer to a different object
- `a = b = c = {};`
- `a`, `b`, and `c` all refer to the same object

Enumeration

- ```
var triangle = { a: 3, b: 4, c: 5,
 color : function () { return "boo"; }
};
```
- ```
function ColoredTriangle() {  
  this.color = "red";  
}  
ColoredTriangle.prototype = triangle;  
– we will talk about this part in a moment
```
- ```
var obj = new ColoredTriangle();
for (var prop in obj) {
 console.log("obj." + prop + " = " + obj[prop] +
 (obj.hasOwnProperty(prop) ? " (own)" : ""));
}
```
- for ... in, sees only enumerable properties (discussed later)

```
obj.color = red (own)
obj.a = 3
obj.b = 4
obj.c = 5
```

# The delete operator

- use to remove a property from an object if it has one
- will not touch any of the objects in the prototype chain
- removing a property from an object may allow a property from the prototype linkage to shine through
- removing nonexistent property cause no error
- `obj.color ↦ "red"`
  - own property
- `delete obj.color`
- `obj.color ↦ function triangle.color()`
  - inherited property becomes visible

# Functions 1/2

- functions in JavaScript are objects
- function `f(x) { return x+1 }`
  - `f.__proto__ === Function.prototype`
  - `f.__proto__.__proto__ === Object.prototype`
  - `f.name`  $\mapsto$  "f"
  - `f.length`  $\mapsto$  1
    - number of arguments
  - `f.prototype` automatically reference to newly created object with only one property: constructor
    - `f.prototype.constructor === f`
    - Why? We will see later.

# Functions 2/2

- since functions are objects, they
  - can be stored in variables, objects and arrays
  - can be passed as arguments to functions
  - can be returned from functions
  - can have properties and methods
- the special thing about functions:
  - they can be invoked

# Function expression

- `function [name]([param[, param[, ... param]]]) {  
 statements  
}`
  - expression whose value is a function
  - optional name
    - name is callable only inside of the function body
    - name is also used by IDEs to identify the function
- `var multiply = function(x, y) { return x * y; };`
- `var multiply = function fn(x, y) { return x * y; };`
  - name `fn` can be used only inside of its body



# Function statement

- `function name([param[, param[, ... param]]) {  
 statements  
}`
- `function multiply(x, y) { return x * y; }`
- equivalent to:  
`var multiply = function multiply(x, y) { return x * y; };`

# Nested functions

- functions can be defined inside of other functions
- an inner function has access to the parameters and variables of all functions it is nested within
- the function object created by a function literal contains a link to that outer context
  - this is called closure
- ```
function f(x) {  
  function g(y) { return x + y; }  
  return g;  
}
```

```
h = f(10) ↦ function g()  
h(7) ↦ 17
```

Function invocation

- besides the declared parameters, every function receives two additional parameters:
 - arguments (array like object containing list of all supplied arguments)
 - this (important in object oriented programming)
- no error occurs when the number of arguments and parameters do not match
 - extra arguments are ignored (but are present in arguments parameter)
 - missing parameters are set to the undefined value
 - no type checking
- there are four invocation patterns:
 - the function invocation pattern
 - the method invocation pattern
 - the constructor invocation pattern
 - the apply invocation pattern
- these patterns differ in how the this is initialized

The function invocation patterns

- if function is **not** invoked using
 - dot expression: `obj.f()`
 - subscript expression: `obj["f"]()`
 - `new`
 - `apply`
- then it is invoked as a function
 - ***this*** is bound to the global object (Window)
- `function f(x) { console.log(this); return x+1; }`
- `f(10) ↦ 11`
 - Window → `about:home`

The method invocation pattern

- method
 - function stored as a property of an object
- if an invocation expression contains a dot expression or subscript expression then the function is invoked as a method
 - this is a reference to the object whose method is invoked
 - method can use this to access the object
 - to retrieve values from the object or modify the object
- ```
function f(x) { console.log(this); return x+1; }
var o = { g: f }
```
- ```
o.g(10) ↦ 11 or o["g"](10) ↦ 11
```

 - `Object { g: f() }`

The constructor invocation pattern

- JavaScript use a prototypal inheritance
- function invocation with the new prefix
 - creates new object with
 - a hidden link (`__proto__`) to the value of the function's prototype member
 - a constructor property set to the function
 - this will be bound to newly created object
- constructors are
 - functions intended to be used with the new prefix
 - by convention, kept in variables with a capitalized name
 - constructor called without the new prefix pollute the global object
 - without a compile-time or runtime warning
 - so the capitalization convention is **really** important
- use of this style of constructor functions is not recommended
 - see slide “[OOP – classically inspired syntax](#)”

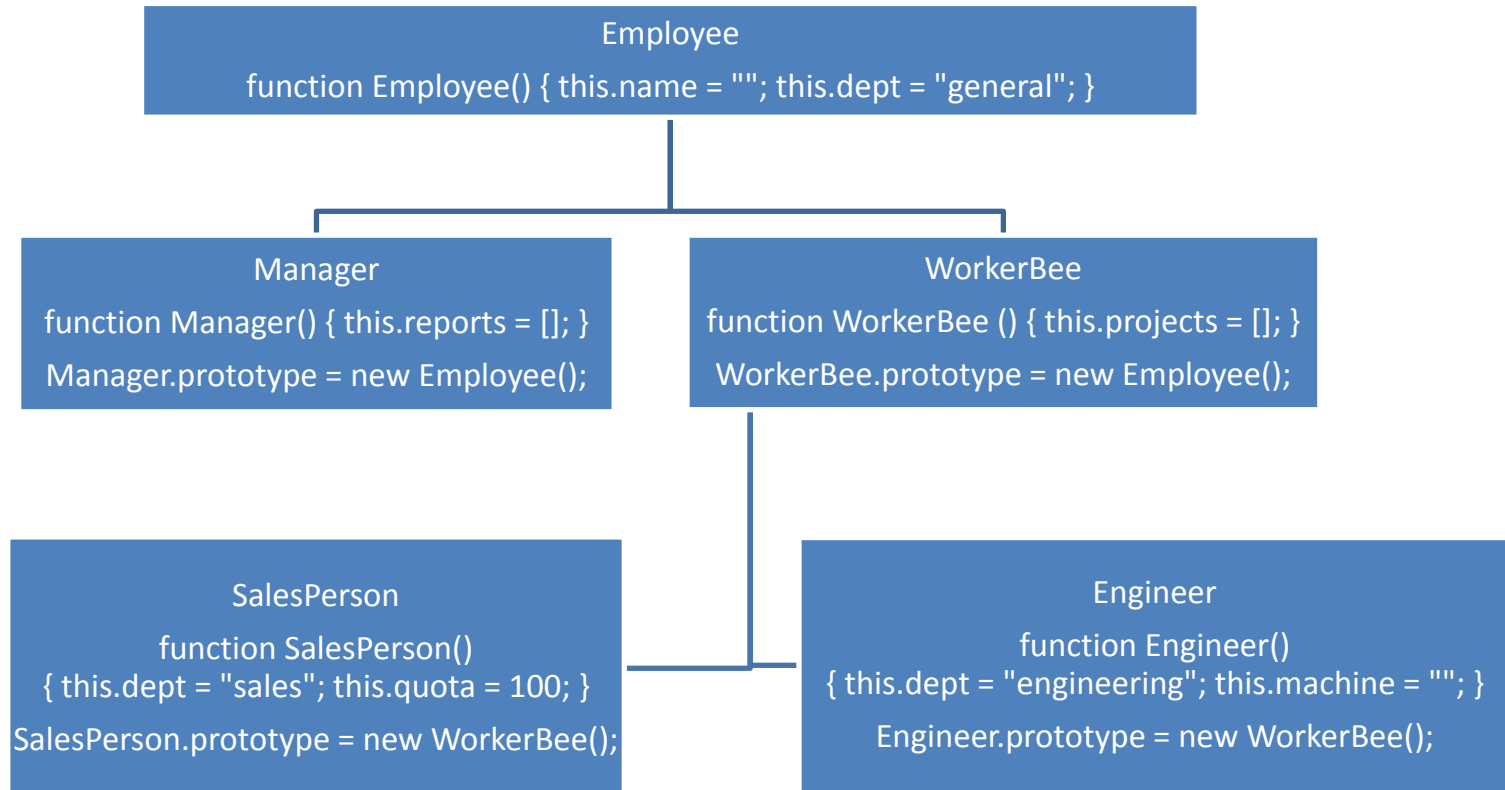
The constructor invocation pattern

- `var Person = function (age) { this.age = age; };`
 - constructor function called Person
 - makes an object with an age property
- `Person.prototype.getAge =`
`function ( ) { return this.age; };`
 - give all instances of Person a public method getAge
- `var janko = new Person(18);`
 - make an instance of Person and set age to 18
- `janko.getAge( ) ↦ 18`

The apply invocation pattern

- `Function.prototype.apply(thisArg, [argsArray])`
 - `thisArg`
 - the value that should be bound to *this*
 - `argsArray`
 - an array of parameters
- each function inherits `apply` method
- `var a = { 0: "a", 1: 1, 2: true, length: 3 }`
- `Array.prototype.join.apply(a, [", "]) ↦ "a, 1, true"`

Pseudoclassical objects



Objects – example usage

- `var e1 = new Engineer();`
 - `Engineer {dept: "engineering", machine: "", projects: Array[0], name: "", dept: "general"}`
- `var e2 = new Engineer();`
 - `Engineer {dept: "engineering", machine: "", projects: Array[0], name: "", dept: "general"}`
- `e2.machine = "linux";`
 - `Engineer {dept: "engineering", machine: "linux", projects: Array[0], name: "", dept: "general"}`
- `e1.projects = [ "p1", "p2" ];`
 - `Engineer {dept: "engineering", machine: "", projects: Array[2], projects: Array[0], name: ""...}`
- `e1.projects // ["p1", "p2"]`
- `e.__proto__`
 - `WorkerBee {projects: Array[0], name: "", dept: "general"}`
- `e.__proto__.projects // []`

Objects – dynamic adding of new prop

- `e1.salary` // undefined
- `WorkerBee.prototype.salary = NaN;`
- `e1.salary` // NaN
- `e2.salary` // NaN

__proto__ vs. prototype

- prototype is the object used to build __proto__ when you create an object with new
 - available only on the constructor functions
 - not the instances themselves (or other objects)
- __proto__ is the object used in the lookup chain
- Warning: ???
 - Changing the __proto__ of an object is a very slow operation in every browser and JavaScript engine

Private properties and methods

- objects cannot have private properties and methods
- BUT: private properties can be added using closures in constructor function as its local variables
- ```
function Container(private_value) {
 var private_counter = 2;
 function private_dec () {
 if (private_counter > 0)
 { private_counter -= 1; return true; }
 else
 { return false; }
 }

 this.getValue = function () {
 return private_dec () ? private_value : null;
 };
}
```
- ```
x = new Container(42);  
x.getValue() ↦ 42; x.getValue() ↦ 42; x.getValue() ↦ null
```

Prototypal objects

- `Object.create(proto[, propertiesObject])`
 - `proto`: the prototype of the newly-created object
 - `propertiesObject`: optional object whose enumerable own properties are merged with the newly-created object
 - without `propertiesObject` equivalent to
 - ```
Object.create = function (o) {
 var F = function () {};
 F.prototype = o;
 return new F();
};
```
- `var janko = Object.create(ferko);`

# Prototypal objects

- ```
var ferko = {  
  name : 'Ferko',  
  get_name : function () { return this.name; },  
  says : function () { return this.saying; }  
};
```
- ```
var janko = Object.create(ferko);
```
- ```
janko.name = 'Janko'; janko.surname = 'Hrasko'; janko.saying = 'Hello!';  
janko.get_name = function () { return this.name + ' ' + this.surname; };
```
- ```
ferko ↦ Object { name: "Ferko", get_name: ferko.get_name(), says:
ferko.says() }
```
- ```
janko ↦ Object { name: "Janko", surname: "Hrasko", saying: "Hello!",  
get_name: janko.get_name() }
```
- ```
janko.says() ↦ "Hello!"
```
- ```
janko.get_name() ↦ "Janko Hrasko"
```
- ```
ferko.get_name() ↦ "Ferko"
```

# Functional objects

- start by making a function that will produce objects
  - give it a name that starts with a lowercase letter
    - because it will not require the use of the *new* prefix
- function contains four steps:
  1. creates a new object
    - some ways to make an object:
      - use object literal
      - call a constructor function with the new prefix
      - make a new instance from an existing object using the Object.create method
      - call any function that returns an object
  2. optionally defines private instance variables and methods
    - these are just ordinary vars of the function
  3. augments new object with methods
    - these methods will have privileged access to the parameters and the vars defined in the second step
  4. returns new object



# Functional objects

```
var constructor = function (spec, my) {
 my = my || {};
 var other_private_instance_variables;

 // add shared vars and functions to my
 my.member = "value";

 var that = {}; // a new object

 // private instance vars and methods

 // Add privileged methods to that
 // (those methods that can access
 // private vars and functions)
 var methodical = function () {
 // ...
 };
 that.methodical = methodical;

 return that;
};
```

- spec object contains all of the information that the constructor needs to make an instance
- my object is a container of secrets that are shared by the constructors in the inheritance chain
- The advantage to defining methodical in two steps is that if other internal methods want to call methodical, they can call methodical() instead of that.methodical().
  - If the instance is damaged or tampered with so that that.methodical is replaced, the methods that call methodical directly will continue to work the same because their private methodical is not affected by modification of the instance.

# Keyword this in different contexts

- global execution context
  - i.e. outside of any function
  - refers to the global object
    - `this === window`  $\mapsto$  true
- function context, simple call
  - not a method of an object
  - function `f(){ "use strict"; return this; }`
    - `f() === undefined`  $\mapsto$  true
    - if not in strict mode, then `this === window`  $\mapsto$  true

# this in private methods – problem

- ```
function Test() {  
    "use strict";  
    function private_f() { return this; }  
    this.getValue = function () {  
        return [this, private_f()];  
    };  
}
```
- ```
x = new Test();
x.getValue() $\mapsto$ [Test, undefined]
```

# this in private methods – solution

- ```
function Test() {  
    "use strict";  
    var that = this;  
    function private_f() { return that; }  
    this.getValue = function () {  
        return [this, private_f()];  
    };  
}
```
- ```
x = new Test();
x.getValue() ↦ [Test, Test]
```

# Keyword this in different contexts

- function context, an object method
  - this is set to the object the method is called on
  - `var o = {prop: 37};`  
`function independent() { return this.prop; }`  
`o.f = independent;`
  - `independent() ↦ undefined`
    - if `independent` uses strict mode, then
      - `TypeError: Cannot read property 'prop' of undefined`
  - `o.f() ↦ 37`
  - it only matters that the function was invoked from the `f` member of `o` (not that it was declared inside of `o`)

# Keyword this in different contexts

- function context, an object method
  - only the most immediate member reference counts
  - `var o = { prop: 37 };`  
`function independent() { return this.prop; }`  
`o.f = independent;`  
`o.b = {g: independent, prop: 42};`
  - `o.f()`  $\mapsto$  37
  - `o.b.g()`  $\mapsto$  42

# Keyword this in different contexts

- function context, an object method
  - method is on an object's prototype chain
    - this refers to the object the method was called on
      - as if the method was on the object
  - `var o = { f: function() { return this.a + this.b; } };`  
`function Subclass() { this.a = 1; this.b = 4; };`  
`Subclass.prototype = o;`  
`p = new Subclass();`
  - `p.f()`  $\mapsto$  5

# Keyword this in different contexts

- function context, an object getter or setter
  - function modulus() {  
    return Math.sqrt(this.re \* this.re + this.im \* this.im);  
}
  - var o = { re: 1, im: -1,  
    get phase() { return Math.atan2(this.im, this.re); }  
};  
Object.defineProperty(o, "modulus", {  
    get: modulus, enumerable: true, configurable: true  
});
  - console.log(o.phase, o.modulus);  
-0.7853981633974483 1.4142135623730951



# Keyword this in different contexts

- function context, an object constructor
  - when a function is used as a constructor (with the new keyword), its this is bound to the new object being constructed
  - `function C() { this.a = 37; }`  
`var o = new C();`
  - `console.log(o.a) ↦ 37`
- the default for a constructor is to return the object referenced by this
  - but it can return some other object instead
- `function C2() { this.a = 37; return {a:38}; }`  
`o = new C2();`
- `console.log(o.a) ↦ logs 38`

# this manipulation – call, apply

- this keyword can be bound to a particular object using the call or apply methods
  - all functions inherit them from Function.prototype
- ```
function add(c, d) {  
  "use strict";  
  return this.a + this.b + c + d;  
}  
var o = { a: 1, b: 3 };
```
- `add(0, 0) ↦ TypeError: Cannot read property 'a' of undefined`
- `add.call(o, 5, 7) ↦ 16`
 - Syntax: `fun.call(thisArg[, arg1[, arg2[, ...]]])`
 - calls fun with a given thisArg as this value and arguments arg1, arg2, ...
- `add.apply(o, [10, 20]) ↦ 34`
 - Syntax: `fun.apply(thisArg[, argsArray])`

this manipulation – bind

- `fun.bind(thisArg[, arg1[, arg2[, ...]]])`
 - returns a new function, that regardless of how it is called, has its `this` keyword set to `thisArg` value
 - `arg1, arg2, ...` precede any provided arguments when the new function is called
 - all functions inherit `bind()` from `Function.prototype`
- ```
function f() { return this.a; }
var g = f.bind({ a: "azerty" });
var o = {a:37, f:f, g:g};
```
- `console.log(g()); // azerty`
- `console.log(o.f(), o.g()); // 37, azerty`

# Applications of bind – setTimeout

- ```
function Container() { this.value = 42; }  
Container.prototype.activate = function () {  
    window.setTimeout(this.msg, 1000);  
    window.setTimeout(this.msg.bind(this), 2000);  
};  
Container.prototype.msg = function () {  
    console.log("Value is " + this.value);  
};
```
- ```
var c = new Container();
c.activate();
```

  - after one and two seconds msg method will be called
  - 1. sec: Value is undefined
  - 2. sec: Value is 42

# Applications of bind – shortcuts

- ```
var unboundSlice = Array.prototype.slice;  
function f() {  
    return unboundSlice.apply(arguments).join(", ");  
}
```
- $f(1, "a", 2) \mapsto "1, a, 2"$
- ```
var slice =
 Function.prototype.apply.bind(unboundSlice);
function f() {
 return slice(arguments).join(", ");
}
```
- $f(1, "a", 2) \mapsto "1, a, 2"$

# Objects getters and setters

- ```
var o = {  
  set current (str) {  
    return this.log[this.log.length] = str;  
  },  
  get latest () {  
    if (this.log.length > 0) {  
      return this.log[this.log.length - 1];  
    } else {  
      return null;  
    }  
  },  
  log: []  
}
```

Objects define property

- `Object.defineProperty(obj, "key", {
 enumerable: false,
 configurable: false,
 writable: false,
 value: "static"
});`

Enumeration of all own properties

- **Object.getOwnPropertyNames**
 - returns an array of all own properties names
 - even those not enumerable
- ```
function listProp(obj) {
 for (name of Object.getOwnPropertyNames(obj)) {
 console.log(name);
 console.log(Object.getOwnPropertyDescriptor(obj, name));
 }
}
```
- **listProp(listProp)**
  - prototype
    - Object { value: Object, writable: true, enumerable: false, configurable: false }
  - length
    - Object { value: 1, writable: false, enumerable: false, configurable: true }
  - name
    - Object { value: "listProp", writable: false, enumerable: false, configurable: true }